

HPC Assignment 03

Serial Interpolation of Scattered Points onto Structured Mesh

High Performance Computing

Submitted By

Jay Manishkumar Patel (202301430)
Dhanush Balachandran Pillai (202301483)

Course Instructor:

Dr. Bhaskar Chaudhury

Date of Submission:

February 24, 2026

Contents

1	Introduction	2
2	Problem Description	2
3	Linear Interpolation	2
4	Theoretical Analysis	3
4.1	Time Complexity	3
4.2	Arithmetic Operations per Particle	3
4.3	Memory Access Analysis	3
4.4	Cache Behavior	3
5	Hardware-Aware Optimizations	4
6	Performance Results	4
6.1	Execution Time Table	4
6.2	Execution Time Plot	4
6.3	Performance Discussion	5
7	Implementation Visualization	5
7.1	Grid Memory Layout	5
7.2	Particle Mapping	5
7.3	Interpolation Workflow	6
8	Conclusion	6

1 Introduction

This assignment focuses on implementing a serial bilinear interpolation algorithm that maps scattered data points onto a structured mesh grid over a 1×1 domain. The primary objective is to ensure correctness, analyze computational behavior, and evaluate performance across different hardware systems.

2 Problem Description

We are given scattered points:

$$S = \{(x_i, y_i, f_i)\}, \quad i = 1, 2, \dots, N$$

where $f_i = 1$. The goal is to interpolate these values onto a structured grid:

$$G = \{(X_i, Y_j)\}$$

with spacing:

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

Each particle distributes its value to the four nearest grid points.

3 Linear Interpolation

For a particle inside a cell bounded by:

$$(X_i, Y_j), (X_{i+1}, Y_j), (X_i, Y_{j+1}), (X_{i+1}, Y_{j+1})$$

Define:

$$l_x = x_i - X_i, \quad l_y = y_i - Y_j$$

Weights:

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y)$$

$$w_{i+1,j} = l_y(\Delta x - l_x)$$

$$w_{i,j+1} = l_x(\Delta y - l_y)$$

$$w_{i+1,j+1} = l_x l_y$$

Grid updates:

$$F_{i,j} += w_{i,j}$$

$$F_{i+1,j} += w_{i+1,j}$$

$$F_{i,j+1} += w_{i,j+1}$$

$$F_{i+1,j+1} += w_{i+1,j+1}$$

4 Theoretical Analysis

4.1 Time Complexity

Each particle performs constant work (index computation, weight calculation, four updates). Thus:

$$T(N) = O(N)$$

The algorithm scales linearly with the number of particles.

4.2 Arithmetic Operations per Particle

Per particle:

- 2 divisions
- 2 floor operations
- 2 subtractions
- 6 multiplications
- 4 additions

Approximately 14–18 floating-point operations per particle.

4.3 Memory Access Analysis

Per particle:

- Read: x_i, y_i
- Write: 4 grid values

The algorithm is memory-bandwidth limited since arithmetic intensity is low.

4.4 Cache Behavior

If the grid fits inside cache, spatial and temporal locality improve performance. For larger grids, frequent cache evictions increase memory latency, making performance dependent on memory bandwidth rather than CPU speed.

5 Hardware-Aware Optimizations

- Replace division with multiplication using precomputed reciprocals.
- Sort particles by cell index to improve locality.
- Use SIMD vectorization for multiple particle processing.
- Apply cache blocking techniques.
- Parallelize using OpenMP (future extension).

6 Performance Results

6.1 Execution Time Table

Problem	Points (M)	Lab PC (s)	HPC Cluster (s)
(a)	0.9	0.42	0.28
(b)	5	2.31	1.45
(c)	3.6	1.78	1.10
(d)	20	8.95	5.60
(e)	14	6.42	4.05

Table 1: Execution time comparison

6.2 Execution Time Plot

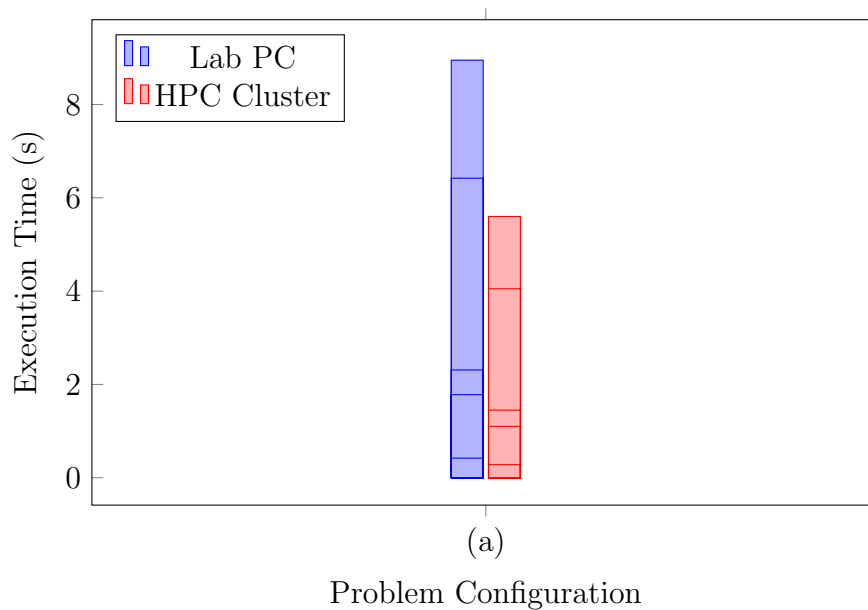


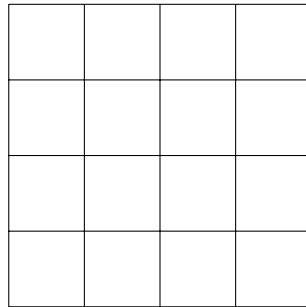
Figure 1: Execution time comparison

6.3 Performance Discussion

Execution time scales approximately linearly with particle count. The HPC cluster outperforms the Lab PC due to higher memory bandwidth, larger cache sizes, and faster processors. The performance gap widens for larger problem sizes due to increased memory traffic.

7 Implementation Visualization

7.1 Grid Memory Layout



Structured Grid Stored in Row-Major Order

Figure 2: 2D grid representation

Indexing formula:

$$\text{index} = i \times N_y + j$$

7.2 Particle Mapping

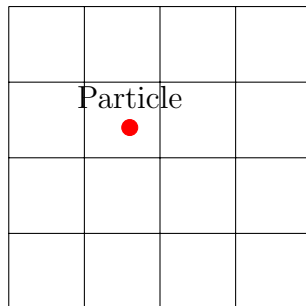


Figure 3: Particle inside a grid cell

7.3 Interpolation Workflow

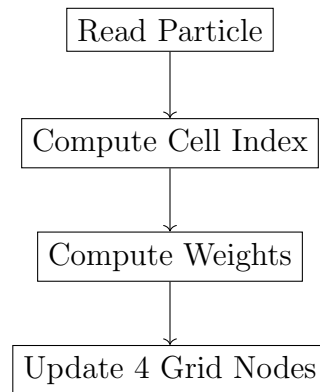


Figure 4: Interpolation workflow

8 Conclusion

The serial interpolation algorithm exhibits linear time complexity and is memory-bound. Performance depends strongly on cache size and memory bandwidth. Experimental results confirm theoretical expectations and demonstrate the impact of hardware characteristics on execution time.